**STUDY NOTES**

**Agile Methods**

Notes on the Scrum Guide and two overviews of agile frameworks

**Sources:** The Scrum Guide 2020, the AltexSoft agile whitepaper, the Agile Methodologies Guide 2025

**These notes:** an original summary written for study. The ideas are the sources'; the wording is not. Short quotations are marked and attributed.

# About these notes

These are study notes on three documents about agile ways of working. Each has its own part below, followed by a part that compares them and a quick-reference section.

| Document | Publisher and date | What it is |
|---|---|---|
| **The Scrum Guide** | Ken Schwaber and Jeff Sutherland, November 2020. 14 pages. | The official definition of Scrum and the authority on its rules. |
| **Agile Project Management: Best Practices and Methodologies** | AltexSoft whitepaper, undated; its sources are from 2016 and 2017. 23 pages. | Agile versus traditional project management, and six frameworks with advice on when to use each. |
| **Agile Methodologies Guide 2025, v1.0** | EuropeanScrum.org, 2025. 26 pages. | Agile at team and organisation level: culture, transformation, four frameworks, structures and tools. |

Everything here is a summary in my own words, written to help a reader recall and apply the ideas. It does not replace the originals. A few short quotations are marked and attributed. Remarks that are mine and not from a source are labelled as such, including the comparison in Part D.

*Licence note:* the Scrum Guide is © 2020 Ken Schwaber and Jeff Sutherland and is offered under the Creative Commons Attribution Share-Alike 4.0 licence. Part A is adapted from it, keeps its terms, and may be shared on the same basis. The other two documents carry no open licence, so Parts B and C are summaries only.

# Ten things to remember

- **Agile is a set of values and principles, not one method.** Scrum, Kanban, Lean, XP and the rest are different ways of putting those values into practice.
- **The 2001 Agile Manifesto ranks four things above their alternatives:** people and interaction, working software, customer collaboration, and responding to change. The alternatives still matter; they just matter less.
- **Scrum rests on empiricism:** make the work visible, inspect it often, and adapt as soon as something is off. Its five events and three artifacts exist to serve those three pillars.
- **A Scrum Team has three accountabilities:** Developers, one Product Owner and one Scrum Master, with no sub-teams and no hierarchy.

- **A Sprint lasts one month or less** and contains Sprint Planning, Daily Scrums, a Sprint Review and a Sprint Retrospective.
- **Each Scrum artifact carries a commitment:** the Product Backlog has the Product Goal, the Sprint Backlog has the Sprint Goal, and the Increment has the Definition of Done.
- **Kanban has no fixed iterations.** It makes work visible on a board, limits how much is in progress, and measures flow.
- **Lean is about removing waste** and delivering more value with less effort. XP is about engineering discipline: test first, refactor, integrate continuously, pair.
- **No framework suits everyone.** Choose by team size, type of product, culture, uncertainty and maturity, and feel free to combine.
- **Changing the method is the easy part.** Both overview documents stress that culture and mindset are where agile adoption succeeds or fails.

# Part A: The Scrum Guide (2020)

The Scrum Guide is the definition of Scrum written by its two creators. It is deliberately brief. It says what Scrum consists of and what each element is for, and leaves techniques to other sources.

## A1. Purpose and definition

*In brief:* Scrum is a lightweight framework for creating value through adaptive solutions to complex problems.

The authors developed Scrum in the early 1990s and first published the Guide in 2010. They warn that altering Scrum's core design, dropping elements or ignoring its rules hides problems and limits the benefit, possibly to nothing. They also note that Scrum has spread well beyond software, so the word “developers” is used to keep things simple, not to exclude other kinds of specialist.

In outline, Scrum needs a Scrum Master to foster an environment in which:

- the Product Owner arranges the work on a complex problem, in order, as a Product Backlog;
- in each Sprint the Scrum Team converts some of that work into an Increment that has value;
- the team and its stakeholders look at the outcome and adjust before the next Sprint;
- and the cycle repeats.
- The framework is **purposefully incomplete**. It defines only the parts needed to put Scrum theory into practice, and relies on the intelligence of the people using it.
- Its rules guide relationships and interactions; they are not detailed instructions.
- Other processes, techniques and methods can be used inside it. Scrum either wraps around existing practices or shows them to be unnecessary.
- Scrum makes the relative effectiveness of current management, environment and working techniques visible, so they can be improved.

## A2. Scrum theory

*In brief:* Scrum is founded on empiricism and lean thinking, and works through three pillars.

- **Empiricism:** knowledge comes from experience, and decisions are based on what is observed.

- **Lean thinking:** reduce waste and concentrate on essentials.
- Scrum uses an **iterative, incremental** approach to improve predictability and control risk.
- It brings together people who between them have all the skills the work needs, and who share or acquire skills as required.

| Pillar | What it means |
|---|---|
| **Transparency** | The process and the work must be visible to those doing the work and those receiving it. Important decisions rest on the perceived state of the three artifacts, so an artifact that is not transparent leads to decisions that reduce value and add risk. |
| **Inspection** | Artifacts and progress toward agreed goals are examined often and carefully, to catch unwanted variances early. The five events provide the rhythm for this. |
| **Adaptation** | When the process drifts outside acceptable limits, or the product is unacceptable, adjust as soon as possible. This is harder when people are not empowered or self-managing. |

The pillars depend on one another: without transparency, inspection misleads; without adaptation, inspection is pointless.

## A3. Scrum values

*In brief:* five values that make the pillars work in practice.

| Value | How the Guide describes it |
|---|---|
| **Commitment** | The team commits to reaching its goals and to supporting one another. |
| **Focus** | The team's attention stays on the Sprint's work, so that it advances as far as it can toward its goals. |
| **Openness** | The work and its difficulties are discussed frankly within the team and with stakeholders. |
| **Respect** | Team members regard each other as capable, independent people, and are treated that way by those they work with. |
| **Courage** | Members are willing to do what is right and to take on difficult problems. |

Decisions, actions and the way Scrum is used should strengthen these values. When the team and the people around it live them, transparency, inspection and adaptation come to life and trust grows.

## A4. The Scrum Team

*In brief:* one small, cross-functional, self-managing team with three accountabilities and a single objective at a time.

- A Scrum Team is one Scrum Master, one Product Owner, and Developers. There are **no sub-teams and no hierarchies**.
- It is **cross-functional**: the members have every skill needed to create value each Sprint.
- It is **self-managing**: the team decides internally who does what, when and how.
- It is typically **ten people or fewer**: small enough to stay nimble, large enough to complete significant work.

- If a team grows too large, it should reorganise into several cohesive Scrum Teams focused on the same product. They then share one Product Goal, one Product Backlog and one Product Owner.
- The team is responsible for everything related to the product, including stakeholder collaboration, verification, maintenance, operation, experimentation and research.
- The **whole team** is accountable for producing a valuable, useful Increment every Sprint.

### The three accountabilities

| Who | Accountable for | Notes |
|---|---|---|
| **Developers** | Creating the plan for the Sprint (the Sprint Backlog); building in quality by keeping to the Definition of Done; adapting the plan each day toward the Sprint Goal; holding each other to account as professionals. | Anyone on the team who works on building some part of each Sprint's usable Increment. The skills needed are broad and vary with the field. |
| **Product Owner** | Maximising the value of the product that results from the team's work. Managing the Product Backlog: developing and communicating the Product Goal, creating and explaining backlog items, ordering them, and keeping the backlog visible and understood. | May delegate the work but stays accountable. Is one person, not a committee. The organisation must respect their decisions; anyone wanting to change the backlog has to persuade the Product Owner. |
| **Scrum Master** | Establishing Scrum as the Guide defines it, by helping everyone understand its theory and practice. The effectiveness of the Scrum Team. | A leader who serves the team and the wider organisation. |

"*The Product Owner is one person, not a committee.*"

— The Scrum Guide, 2020, p. 6

### How the Scrum Master serves

| Serves | By |
|---|---|
| **The Scrum Team** | Coaching the team to manage itself and to work across disciplines; keeping the team's attention on valuable Increments that satisfy the Definition of Done; getting impediments removed; making sure every event happens, is positive and productive, and stays within its timebox. |
| **The Product Owner** | Helping find techniques for defining the Product Goal and managing the backlog; helping the team see why backlog items must be clear and concise; helping establish empirical product planning; facilitating stakeholder collaboration when asked or needed. |
| **The organisation** | Leading, training and coaching its adoption of Scrum; planning and advising on implementations; helping staff and stakeholders understand an empirical approach to complex work; removing barriers between stakeholders and Scrum Teams. |

**Takeaway** Scrum names accountabilities, not job titles. Value belongs to the Product Owner, the Sprint plan and quality to the Developers, and the team's effectiveness to the Scrum Master.

## A5. Scrum events

*In brief:* the Sprint contains four other events. Each is a formal chance to inspect and adapt, and together they reduce the need for other meetings.

| Event | Purpose | Who | Timebox |
|---|---|---|---|
| **The Sprint** | The container in which ideas are turned into value. | The whole Scrum Team | Fixed length of one month or less. The next starts immediately. |
| **Sprint Planning** | Lay out the work for the Sprint. | The whole Scrum Team; others may be invited to advise. | At most eight hours for a one-month Sprint. |
| **Daily Scrum** | Inspect progress toward the Sprint Goal and adjust the Sprint Backlog. | The Developers | Fifteen minutes, same time and place every working day. |
| **Sprint Review** | Inspect the outcome of the Sprint and decide what to adapt next. | The Scrum Team and key stakeholders | At most four hours for a one-month Sprint. |
| **Sprint Retrospective** | Work out how to raise quality and effectiveness. | The Scrum Team | At most three hours for a one-month Sprint. |

For shorter Sprints the planning, review and retrospective are usually shorter too. Holding events at the same time and place reduces complexity. Skipping or changing an event loses an opportunity to inspect and adapt.

### The Sprint

*"Sprints are the heartbeat of Scrum, where ideas are turned into value."*

— The Scrum Guide, 2020, p. 7

During a Sprint:

- no change is made that would put the Sprint Goal at risk;
- quality does not go down;
- the Product Backlog is refined as needed;
- as understanding grows, scope can be clarified and renegotiated with the Product Owner.
- Sprints give predictability by guaranteeing inspection and adaptation at least once a calendar month. A longer horizon risks an invalid Sprint Goal, more complexity and more risk. Shorter Sprints produce more learning cycles and limit exposure.
- Burn-down charts, burn-up charts and cumulative flow diagrams can help forecast progress, but they do not replace empiricism: in complex work, only what has already happened can inform decisions about the future.
- A Sprint may be cancelled if its goal becomes obsolete. **Only the Product Owner** can cancel it.

### Sprint Planning: three topics

| Topic | What happens |
|---|---|

| | |
|---|---|
| **1. Why is this Sprint valuable?** | The Product Owner suggests ways the product's value and usefulness could grow during the Sprint. The whole team then defines a Sprint Goal that tells stakeholders why the Sprint matters. It must be settled before planning ends. |
| **2. What can be Done this Sprint?** | In discussion with the Product Owner, the Developers select Product Backlog items, refining them if useful. Knowledge of past performance, upcoming capacity and the Definition of Done makes their forecast more reliable. |
| **3. How will the work get done?** | For each selected item the Developers plan the work needed to meet the Definition of Done, often breaking items into pieces of a day or less. How they do this is entirely up to them. |

The Sprint Goal, the selected items and the plan for delivering them together form the Sprint Backlog.

### Daily Scrum

- It is an event for the Developers. A Product Owner or Scrum Master who is actively working on Sprint Backlog items takes part as a Developer.
- The Developers may use any structure and technique they like, provided the meeting concentrates on progress toward the Sprint Goal and produces a workable plan for the next day.
- It improves communication, exposes impediments, speeds up decisions and removes the need for other meetings.
- It is not the only time the plan may change. Developers often meet during the day for more detailed discussion.

### Sprint Review

- The team shows key stakeholders what it produced, and together they discuss how far the product has moved toward the Product Goal.
- Together they look at what was accomplished and what has changed in their environment, then work out what to do next. The Product Backlog may be adjusted.
- It is a working session. The Guide advises against reducing it to a presentation.
- Only the retrospective follows it in the Sprint.

### Sprint Retrospective

- The team inspects how the Sprint went with regard to individuals, interactions, processes, tools and the Definition of Done.
- It discusses what went well, what problems arose, and how they were or were not solved, and looks for the origins of assumptions that led it astray.
- It picks the most helpful changes. The most significant are addressed as soon as possible and may be added to the next Sprint Backlog.
- It concludes the Sprint.

**Takeaway** Every event has a purpose and a maximum length. The Daily Scrum belongs to the Developers, and only the Product Owner can cancel a Sprint.

## A6. Scrum artifacts and their commitments

*In brief:* three artifacts represent work or value, and each has a commitment against which progress can be measured.

| Artifact | What it is | Its commitment |
|---|---|---|
| **Product Backlog** | An ordered list, never complete, of everything needed to make the product better. All of the Scrum Team's work comes from it. | Product Goal: a future state of the product that the team can plan against. It is the long-term objective and sits in the Product Backlog. |
| **Sprint Backlog** | The Sprint Goal (why), the items selected for the Sprint (what) and a workable plan for delivering the Increment (how). A plan made by and for the Developers, updated throughout the Sprint. | Sprint Goal: the single objective for the Sprint. It gives coherence and focus while leaving room in the exact work needed to reach it. |
| **Increment** | A concrete step toward the Product Goal. It adds to all earlier Increments, is thoroughly verified, and must be usable. | Definition of Done: a formal statement of the quality an Increment must reach to be acceptable for the product. |

### Product Backlog and Product Goal

- **Refinement** is the continuing activity of breaking items down and defining them more precisely, adding details such as a description, order and size.
- Items that the team can complete within one Sprint are considered ready for selection at Sprint Planning.
- The Developers who will do the work are responsible for sizing it. The Product Owner can help them understand trade-offs.
- The Guide defines a **product** as a vehicle for delivering value, with a clear boundary, known stakeholders and well-defined users or customers. It can be physical, a service, or something less tangible.
- The team must fulfil or abandon one Product Goal before taking on the next.

### Sprint Backlog and Sprint Goal

- The Sprint Backlog shows, openly and up to date, what the Developers intend to do in the Sprint, in enough detail for progress to be inspected at the Daily Scrum.
- The Developers commit to the Sprint Goal. It encourages the team to work together instead of on separate initiatives.
- If the work turns out differently from what was expected, the Developers negotiate the scope of the Sprint Backlog with the Product Owner, without affecting the Sprint Goal.

### Increment and Definition of Done

- Several Increments may be created within one Sprint. Their sum is presented at the Sprint Review.
- An Increment may be delivered to stakeholders before the Sprint ends. The Sprint Review is never a gate for releasing value.
- As soon as a backlog item satisfies the Definition of Done, a new Increment exists.

- An item that does not meet the Definition of Done cannot be released, or even shown at the Sprint Review. It goes back to the Product Backlog.
- If the organisation has a standard Definition of Done, every Scrum Team follows it as a minimum. If not, the team creates one suited to the product.
- Several Scrum Teams working on one product must define and follow the same Definition of Done.

**Takeaway** Ask of any piece of Scrum work: which goal does it serve, and does it meet the Definition of Done? If it is not Done, it is not part of the Increment.

### A7. End note and history

- Scrum is free. The Guide calls the framework **immutable**: implementing only parts of it is possible, but the result is not Scrum.
- Scrum exists only as a whole and works well as a container for other techniques, methods and practices.
- Schwaber and Sutherland first presented Scrum together at the OOPSLA conference in 1995.
- The Guide credits early collaborators and the first organisations where Scrum was tried: Individual Inc., Newspage, Fidelity Investments and IDX (now GE Medical).

## Part B: Agile Project Management whitepaper

This whitepaper from the consultancy AltexSoft introduces project management, contrasts the traditional waterfall model with agile, and then describes six frameworks, each with a short “when to use” section. It cites survey figures throughout, mainly from 2016 and 2017 reports.

### B1. Project management and its phases

*In brief:* any project follows a controlled sequence of phases, but that generic structure is too broad to manage real work.

- The paper opens with Gartner's definition of project management: applying knowledge, skills, tools and techniques to project activities so that the project's requirements are met.
- It reports a Project Management Institute (PMI) survey finding that only about half of organisations (54%) fully appreciate the value of project management practices.
- It adds that organisations using proven practices waste far less money and complete more successful projects.
- A successful project is one finished on time and on budget that also delivers the expected benefits.

According to PMI, a typical project passes through five phases:

- Initiation
- Planning
- Execution
- Performance and monitoring
- Project close

These phases make up the project management lifecycle. Within each there are internal stages that vary with the scope, the team, the industry and the project, which is why many different techniques and methodologies have been developed.

## B2. Traditional methodologies and the waterfall model

*In brief:* the linear approach works for stable, well-defined projects but is slow and inflexible for software.

- Traditional methods take the project through its phases one after another, in strict order. A stage does not start until the one before is finished.
- The approach is most common in construction and manufacturing, where little changes between stages, and was carried over into software engineering.
- In software it is known as the **waterfall model**, dominant since the early 1970s when Winston W. Royce formally described it.
- It puts heavy weight on planning and specification, which the paper says can take up to 40% of a project's time and budget.
- **Where it fits:** clearly defined projects with a single deliverable and a fixed deadline. It needs thorough planning, extensive documentation and tight control, and in theory gives on-time, on-budget delivery with low risk.
- **Where it fails:** applied to real software work it tends to be slow, costly and rigid. Because it cannot adjust the product to changing market needs, it often wastes resources and can end in failure.

## B3. The agile approach

*In brief:* agile replaces one long linear pass with many short cycles, each a small project in itself.

### Origins

- The paper traces incremental development back to 1957, when a group of engineers used such techniques on software for IBM and Motorola without having a name for them.
- The modern approach dates from 2001, when seventeen software professionals met to discuss alternatives and wrote the **Manifesto for Agile Software Development**.
- It reports that 94% of organisations were practising agile in 2015.

### The four values of the Manifesto

| Valued more | Valued less |
|---|---|
| Individuals and interactions | Processes and tools |
| Working software | Comprehensive documentation |
| Customer collaboration | Contract negotiation |
| Responding to change | Following a plan |

The Manifesto is completed by twelve principles, which the paper mentions but does not list.

### How agile work is organised

- The project is divided into small cycles, called sprints in Scrum. Each has a backlog and includes design, implementation, testing and deployment for an agreed scope.
- The paper's diagram shows a loop: product backlog, sprint backlog, then a sprint of planning, design, development, testing and release, leading toward the final product.
- At the end of each sprint a potentially shippable increment is delivered, so the product grows gradually and features are validated early, which lowers the chance of delivering something that fails.

### Five characteristics

| Characteristic | Meaning |
|---|---|
| **Flexibility** | The scope of work can change as new requirements appear. |
| **Work breakdown** | The project consists of small cycles. |
| **Value of teamwork** | Team members work closely together and each has a clear view of their responsibilities. |
| **Iterative improvement** | Work done in a cycle is reassessed frequently to make the final product better. |
| **Cooperation with the client** | The customer is closely involved and can change requirements or accept the team's suggestions. |

Benefits reported in the survey the paper cites: ability to manage changing priorities (88% of respondents), higher team productivity (83%) and better project visibility (83%).

**Takeaway** Agile is an umbrella term. What the frameworks share is short cycles, close teamwork, customer involvement and a willingness to change scope.

## B4. Six frameworks side by side

*In brief:* the paper's own summary table, condensed, with its adoption figures and its advice on when each framework fits.

| Framework | Key features | When to use | Share of organisations using it |
|---|---|---|---|
| **Scrum** | Work is cut into sprints of one to four weeks. The team follows a plan for each sprint. People have predefined roles. | Long-term, complex projects where stakeholder feedback may change requirements, the amount of work cannot be estimated exactly, and the release date is not fixed. | 58% exclusively; a further 18% combined with other techniques |
| **Kanban** | Built on visualising the workflow. Work in progress is prioritised and limited. No timeboxed cycles. The plan can change at any time. | Small releases and shifting priorities; work that arrives in small pieces, such as fixing defects after the main release. | 43% |

| | | | |
|---|---|---|---|
| **Hybrid** | Agile and waterfall used together. Agile development under waterfall conditions: fixed deadline, forecast budget, thorough risk assessment. | Products that depend on both hardware and software, or customers who will not accept an open-ended budget and timeframe. | 23% |
| **Bimodal** | Two separate modes and teams: Mode 1 (traditional) keeps core IT systems stable; Mode 2 (agile) delivers innovative applications. Collaboration between them matters. | Companies with both long-term and short-term projects needing different management approaches. | 16% |
| **Lean** | Fast development with less effort, time and cost. The shortest possible cycle. Early delivery followed by continuous improvement. An independent team with wide responsibilities. | Small, short-term projects; minimum viable products; customers able to give continuous feedback; a team in one office. | 17% |
| **Extreme Programming (XP)** | Focus on the technical side. Engineering practices for clean code. Face-to-face communication and customer involvement are essential. | New systems built under tight deadlines by small teams (no more than twelve) where customers and managers can work with the developers. | 9%; 10% use a Scrum/XP hybrid |

## B5. Scrum as the whitepaper describes it

*In brief:* three roles, sprints, three artifacts and four recurring meetings. Note that this account follows the 2016 Scrum Guide and differs in places from the 2020 edition in Part A; Part D lists the differences.

- Scrum was first described in 1986 by Hirotaka Takeuchi and Ikujiro Nonaka in an article on new product development, and formulated almost a decade later. Schwaber and Sutherland presented it at OOPSLA in 1995.
- It predates the Agile Manifesto but is consistent with its values.

| Role | Description in the paper |
|---|---|
| **Scrum Master** | The central figure, whose main responsibility is removing obstacles that keep the team from working efficiently. |
| **Product Owner** | Usually a customer or other stakeholder, involved throughout, who conveys the overall vision and gives feedback after every sprint. |
| **Scrum Team** | A cross-functional, self-organising group responsible for building the product, of up to seven members. |

- A **sprint** is one to four weeks. Longer iterations lose the predictability and flexibility that are Scrum's main benefits. All sprints in a project should be the same length.
- The **Product Backlog** is an ordered list of items that might be needed in the product and the single source of requirements. It is updated as requirements, fixes and features change.

- The **Sprint Backlog** is the list of tasks the team must complete to deliver an increment of working software by the end of the sprint.
- The **Sprint Burndown Chart** shows the work remaining in a sprint day by day, and helps predict whether the sprint goal will be met on time.

The four meetings are the Daily Scrum (fifteen minutes, same place and time, planning the next 24 hours), Sprint Planning (what can be done and how; up to eight hours for a one-month sprint), the Sprint Review (the team shows completed work and everyone decides what to do next; four hours for a one-month sprint) and the Sprint Retrospective (what went well or badly and how to improve; three hours for a one-month sprint).

The paper reports that Scrum has the trust of 89% of agile users, that large technology companies use it, that it has spread to finance, consulting and entertainment, and that 98% of respondents to the 2016 State of Scrum report intended to keep using it.

## B6. Kanban

*In brief:* visualise the workflow, limit work in progress, and pull the next item when one is finished.

- Kanban comes from a visual card system used at Toyota to control production. The word is translated in the paper as "visual signal".
- It limits work in progress (WIP) to match the team's capacity. When a task is done, the team takes the next from the pipeline.
- Benefits listed: more flexible planning, faster turnaround, clear objectives and transparency.
- Unlike Scrum, it requires no standard procedures and no fixed iterations.
- The **Kanban board** in the paper has three columns: backlog, in progress, completed. It may be sticky notes on a whiteboard or an online tool such as Trello.
- Each card says who is responsible for an item, what the task is and when it is due, and can carry comments and attachments. Seeing everyone's contribution helps the team focus on finishing work well and on time.
- Some organisations (8%) combine Scrum and Kanban as **Scrumban**.

## B7. Hybrid

*In brief:* use waterfall where planning matters and agile where flexibility matters.

- Agile is iterative and flexible; waterfall is step by step, needs careful planning and resists change. Some organisations find that combining them works better than choosing one.
- A common split: waterfall for budgeting, planning and hardware set-up; agile for software development and testing.
- Alternatively, agile practices can be embedded in a waterfall process: planning in sprints, testing built into development, regular feedback, Kanban boards, retrospectives.
- Which features to combine depends on the project.

## B8. Bimodal

*In brief:* run two distinct styles of work in parallel, one for predictability and one for exploration.

- Gartner introduced the term “Bimodal IT” in 2014.
- **Mode 1** is traditional and suits well-understood, predictable areas. Its team develops and maintains the applications and core systems that support long-term business needs.
- **Mode 2** is exploratory and nonlinear, built for solving new problems quickly. Its team delivers innovative applications often, to win new customers and meet short-term needs, and may change the product after feedback.
- The modes need different skills, techniques and tools, so two separate groups are used. They report through different structures but must communicate.
- Each depends on the other: Mode 2 uses the information and services that Mode 1 provides, and Mode 1 relies on Mode 2 to test new product ideas and development methods.

## B9. Lean

*In brief:* deliver more value with less effort, investment and time by removing whatever does not add value.

Lean shares its origins with Kanban in the Toyota Production System. Mary and Tom Poppendieck applied it to software in their book *Lean Software Development*, with seven principles:

| Principle | Meaning |
|---|---|
| **Eliminate waste** | Waste is anything that adds no value to the project. In software that includes idle time, unneeded features and defects. |
| **Amplify learning and create knowledge** | Development is a continuous learning process. Developers rarely get code right first time and learn by solving problems and producing variations. |
| **Decide as late as possible** | Later decisions rest on more facts. With technology dating quickly, it is wise to delay irreversible design choices and keep capacity for change. |
| **Deliver as fast as possible** | Short cycles bring feedback sooner and let the customer postpone final decisions until they know more. |
| **Empower the team** | Developers understand the details of their work best and should make the technical decisions. |
| **Build integrity and quality in** | Perceived integrity: the customer finds the software has what they need and is easy to use. Conceptual integrity: the architecture is coherent, and the software is usable, fit for purpose, maintainable and extensible. |
| **See the whole** | Engineers take responsibility for the efficiency of the whole system, not only their own part. |

- Lean is iterative and incremental, and delivers a working increment early like other agile approaches.
- What sets it apart is that the team is not bound to formal processes such as recurring meetings or detailed task prioritisation.
- It supports the **minimum viable product (MVP)** technique: release a product with just enough features for early users, then use their feedback to plan what comes next.

- The paper notes that Lean's popularity fell between 2015 and 2016, though it remained among the five most used agile frameworks.

## B10. Extreme Programming (XP)

*In brief:* a small set of engineering practices carried out to an extreme degree, to improve quality and adaptability.

Kent Beck, one of the Manifesto's signatories, created XP in the 1990s while working on a payroll system for Chrysler, and described it in a 1999 book. The paper covers its four most used practices:

| Practice | What it involves |
|---|---|
| **Test-driven development (TDD)** | Write an automated unit test before the code. The test fails at first; then write code to make it pass; then improve the code. This is the red, green, refactor cycle. |
| **Refactoring** | Continually simplify and clarify the code without changing its behaviour, removing clutter and duplication so the code stays easy to maintain and extend. |
| **Continuous integration (CI)** | Commit new code several times a day, with automated testing and deployment. Integration problems are caught early and the main line stays clean and ready to deploy. |
| **Pair programming** | Two engineers work together: one writes the code while the other watches, suggests and navigates. |

- **Benefits claimed for TDD:** constant feedback on every component and so higher quality; unit tests that act as always-current documentation; a team that must think critically about the intended outcome; and less time spent debugging.
- **Pair programming, for and against:** the pair is expected to produce better design with fewer mistakes and to spread knowledge across the team. The cost is short-term productivity; the research cited puts the extra time per task at 15 to 60 percent, which supporters argue is repaid by higher quality.
- XP relies on short iterations, which the paper gives as no longer than a week, strong customer involvement, the simplest possible design, and unit testing skills.
- Many teams take XP's technical practices and combine them with Scrum's management structure.

## B11. Conclusion and reading notes

The paper concludes that agile is a way of thinking about managing software development, and that higher speed, flexibility and productivity are why organisations adopt it. It reports that 98% of companies have had success with agile projects. It also allows that, depending on goals and requirements, waterfall or a combination may still be the better choice.

### Reading notes (mine)

- **The statistics are dated.** They come from surveys of roughly 2015 to 2017. Treat them as a snapshot of that period, not of today.

- **The Scrum description is older than the 2020 Guide.** Team size, role names, artifacts and the planning questions have all changed since.
- **There are editing slips in the PDF.** The contents page swaps the "when to use" labels for Bimodal and Lean. One figure about wasted money is printed without its unit ("28 less"), most likely meaning 28 times less. Some paragraphs are repeated.

**Takeaway** Use this paper for the comparison of frameworks and the "when to use" advice. Use the Scrum Guide for what Scrum actually requires.

# Part C: Agile Methodologies Guide 2025

This guide from EuropeanScrum.org is the broadest of the three. It covers agile as a culture, how an organisation transforms, four frameworks, two related management ideas, how to choose a method, the structures that support agile at scale, and common software tools. Nearly every section ends with a short illustrative example.

## C1. What agile is, and why

*In brief:* an iterative, incremental approach that puts flexibility, collaboration and continuous delivery of value ahead of following a long-term plan.

- Agile arose at the start of the 2000s in reaction to traditional project management, which was rigid and slow to adapt. Its formal start was the 2001 Agile Manifesto, with four values and twelve principles.
- It is no longer limited to software. The guide lists manufacturing, marketing, human resources, education and finance among the fields now using it.
- Organisations that adopt it well are described as more flexible, quicker to respond to the market, and consistently focused on customer satisfaction.
- Its value is said to lie not only in faster delivery but in better quality, lower risk and higher productivity, through collaboration, adaptation and frequent feedback.

### The four values, as the guide explains them

The guide stresses that the Manifesto does not abolish processes, tools, plans or documentation. It reorders priorities toward what creates value under uncertainty.

| Value | The guide's explanation |
|---|---|
| **Individuals and interactions over processes and tools** | Effective teams depend more on communication, trust and collaboration than on rigid process or sophisticated tools. Processes serve people, not the reverse. |
| **Working software over comprehensive documentation** | The aim is solutions that work and bring tangible value. Documentation is kept, but cut to what is needed for efficiency, shared knowledge and traceability. |
| **Customer collaboration over contract negotiation** | The customer is brought into development through trust and constant dialogue. Value is created jointly instead of being fixed in advance by a contract. |

| | |
|---|---|
| **Responding to change over following a plan** | Projects do not happen in stable conditions, so planning is continuous and flexible. Change is treated as a chance to improve. |

"*The customer is not a spectator, but an active part of the team.*"

— Agile Methodologies Guide 2025, p. 6

### Principles the guide highlights

It does not list all twelve principles. It picks out seven themes:

- Deliver value early and continuously, so the customer can validate and respond sooner.
- Accept changing requirements, even late, when they add value.
- Business people and developers work together daily.
- Trust motivated people, and give them autonomy and a suitable environment.
- Prefer face-to-face conversation, even with digital tools available.
- Keep things simple: maximise the amount of work not done.
- Inspect and adapt continuously, through retrospectives and iterative improvement.

### Agile culture

The guide insists that agile culture is more than applying Scrum or Kanban. It is a way of thinking and collaborating, with five traits:

| Trait | What it looks like |
|---|---|
| **Change and adaptation as a mindset** | Change is treated as constant, not as a threat. Uncertainty is handled with short cycles, continuous review and experimentation. |
| **Collaboration and shared responsibility** | Self-managed teams with autonomy and collective responsibility for results. Leadership is shared and situational instead of rigidly hierarchical; communication is open at all levels; failure followed by quick learning is allowed. |
| **Constant feedback and continuous improvement** | Regular retrospectives where the team reflects on what worked, identifies obstacles, and agrees concrete actions for the next cycle. |
| **Transparency as the basis of trust** | Relevant information is visible to everyone, which aligns teams, reduces misunderstanding and builds a sense of purpose. Kanban boards, burndown charts and metrics dashboards help. |
| **Learning culture and growth mindset** | People are encouraged to be curious, teach themselves, mentor one another and experiment. Failure is acceptable when it becomes learning. |

**Takeaway** Agile reorders priorities; it does not throw away plans or documentation. The culture matters more than the framework.

## C2. Agile transformation

*In brief:* adopting agile means changing mindset and culture across the organisation, not only processes and tools.

**Four key components**

- **Cultural change.** The hardest part. The organisation moves from a hierarchical, controlling style to a distributed, collaborative one in which teams have autonomy and leaders facilitate.
- **Training.** Teams and leaders learn the principles and practices, and also the habit of continuous improvement and experimentation.
- **Organisational restructuring.** Teams become multidisciplinary and self-managed, with barriers between departments removed so that a team has all the skills to finish work without outside approvals.
- **An iterative approach.** Short delivery cycles, incremental value, and decisions based on feedback from each iteration.

**Four stages**

| Stage | What happens |
|---|---|
| **1. Exploration and awareness** | Assess readiness: review current processes, find bottlenecks, and confirm that management is committed. |
| **2. Initial adoption (pilot)** | A small group adopts agile practices and serves as a model that demonstrates the benefits to others. |
| **3. Scaling** | After a successful pilot, agile is extended to other teams and departments. Scaling frameworks such as SAFe or LeSS coordinate several teams working on one product or service. |
| **4. Continuous optimisation** | There is no end point. Processes and structure keep being adjusted as customer needs and market conditions change. |

The guide also separates **agile transformation**, which changes how the organisation works internally, from **business agility**, the wider ability of a whole company to adapt to its market (see C8).

## C3. Scrum, as this guide describes it

*In brief:* short iterative cycles that deliver working increments, with constant inspection and adaptation. The outline broadly matches Part A; differences are listed in Part D.

| Component | Description in the guide |
|---|---|
| **Sprints** | Fixed-length cycles, usually one to four weeks, each ending with an increment that is potentially ready to launch or show to the customer. |
| **Product Backlog** | A prioritised list of all features, improvements and fixes, managed and continuously updated by the Product Owner. |
| **Sprint Backlog** | The subset chosen at Sprint Planning: the committed items, the tasks needed to complete them, and a clear sprint goal. The development team owns it and may change it as the sprint goes on. |
| **Sprint Planning** | The Product Owner presents prioritised items, the team selects what it believes it can finish, tasks are defined and the Sprint Goal is set. |

| | |
|---|---|
| **Daily Scrum (stand-up)** | About fifteen minutes. Each member says what they did yesterday, what they will do today, and what is in their way. |
| **Sprint Review** | The team shows the increment to stakeholders and collects feedback, checks whether the goal was met, and adjusts the backlog. |
| **Backlog refinement** | A continuing activity, also called grooming, and not an official Scrum event: clarifying requirements, estimating effort, splitting large items, removing or reordering items. |
| **Sprint Retrospective** | Held after the review and focused on how the team works, not on the product: what to keep, what to fix, and concrete actions for the next sprint. |

- **Product Owner:** maximises product value by managing and prioritising the backlog, acting as the voice of the customer.
- **Scrum Master:** a facilitator, not a hierarchical leader. Helps the team follow Scrum, removes impediments and protects the team from outside interference.
- **Development team:** cross-functional and self-managed, with a mix of skills such as development, testing and design, responsible for delivering a working increment.

## C4. Kanban

*In brief:* continuous flow of work through visible stages, with limits on how much is in progress.

### Four principles

- **Visualise work.** Put all work on a board divided into states such as To Do, In Progress and Done. More columns can be added to match the process.
- **Limit work in progress (WIP).** Cap how many tasks may be under way at once, so the team finishes before starting. This shortens cycle time.
- **Manage flow.** Aim for tasks to complete quickly without overload, and measure flow.
- **Improve continuously.** Use flow data to find bottlenecks, then adjust WIP limits, the board, or working practices.

### Three metrics

| Metric | Definition |
|---|---|
| **Cycle time** | Time from when work on a task starts to when it is completed. |
| **Lead time** | Time from when a task is requested to when it is delivered, including time spent waiting to be started. |
| **Throughput** | Number of tasks completed in a given period. |

Kanban defines no specific roles. The guide suggests a **Service Delivery Manager** to oversee the process and improve flow, and possibly a Product Owner to prioritise the backlog.

## C5. Lean

*In brief:* remove waste and maximise customer value across the whole production system. (The guide's heading for this section is printed as “Read”, evidently a translation slip for Lean.)

Lean comes from the Toyota Production System. Unlike other agile methods it is not only about incremental delivery of a product; it applies to any organisation that wants to improve its operations.

### Six principles

- **Eliminate waste (muda):** remove any activity that gives the customer no direct value.
- **Continuous improvement (kaizen):** small, regular, incremental changes, with every team member involved in spotting problems and suggesting improvements.
- **Create value for the customer:** value is whatever the customer is willing to pay for.
- **Continuous flow:** work should move smoothly through the system, with bottlenecks removed and every step adding value.
- **Pull, not push:** each step takes only what it needs when it needs it, which reduces excess inventory and waiting.
- **Respect for people:** the people doing the work know it best, so they are given the authority to raise problems and suggest fixes.

### The seven wastes

| Waste | Meaning |
|---|---|
| **Overproduction** | Making more than the customer needs. |
| **Waiting** | Delays that leave people or work idle. |
| **Unnecessary transport** | Moving materials or products without need. |
| **Overprocessing** | Doing more work than is required. |
| **Unnecessary inventory** | Holding large stocks of materials. |
| **Unnecessary movement** | Needless motion by people. |
| **Defects** | Errors that require rework. |

### Four tools

- **Value stream mapping:** draw the whole production process to see where time is lost, costs arise or defects enter.
- **5S:** organise the workplace in five steps: sort, set in order, shine, standardise, sustain.
- **Just-in-time:** materials and products arrive exactly when needed, cutting inventory and storage cost.
- **Andon:** a visual signal that lets workers flag a problem as it happens so it can be fixed before it spreads.

## C6. Design Thinking

*In brief:* a user-centred way of solving complex problems through empathy, idea generation and quick experiments.

Where other agile methods concentrate on fast, continuous delivery, Design Thinking concentrates on working out what to build.

| Phase | What the team does |
|---|---|
| **1. Empathise** | Understand users' needs, wishes and behaviour by observing and talking to them. Interviews, surveys and qualitative research are central. |
| **2. Define** | Synthesise what was learned into a clear, concise statement of the problem, centred on the user's needs. Everything later builds on this. |
| **3. Ideate** | Generate many possible solutions with brainstorming, mind mapping and similar techniques, in an atmosphere free of judgement. |
| **4. Prototype** | Turn the best ideas into simple, tangible versions that can be tested cheaply. |
| **5. Test** | Show the prototype to users, gather feedback, and refine or rethink the solution. |

- **Principles:** stay centred on real user needs instead of assumptions; iterate constantly, since the process is non-linear and teams may go back to any phase; collaborate across disciplines; experiment, treating mistakes as learning.
- **Tools:** personas (fictional users based on real data; the guide prints this as “People”), user journey maps (the whole experience of using a product, showing pain points and opportunities) and rapid low-fidelity prototypes.

## C7. Management 3.0 and Culture 3.0

*In brief:* two ideas about leadership and culture that extend agile thinking beyond delivery teams.

### Management 3.0

A management philosophy holding that leaders should manage the system people work in, not the people themselves. Its aim is self-organisation, intrinsic motivation and continuous improvement, with leaders acting as facilitators.

- **Manage the system, not the people:** design an environment in which people can decide and be accountable.
- **Motivate people:** rely on intrinsic motivators such as recognition, purpose and personal development more than on money or status. The guide links this to Daniel Pink's autonomy, mastery and purpose.
- **Delegate decision-making** to teams so they need not wait for approval.
- **Foster creativity and innovation** by giving teams time and space to experiment and learn from failure.
- **Improve continuously,** with leaders facilitating retrospectives.

| Tool | Use |
|---|---|
| **Delegation Poker** | Helps leaders and teams agree how far each kind of decision is delegated, on a scale from the leader deciding everything to the team having full autonomy. |
| **Team Competence Matrix** | Maps team members' skills to reveal gaps and plan development. |
| **Kudo Cards** | A simple way for colleagues to thank each other for good work, supporting intrinsic motivation. |

### Culture 3.0

The guide describes three generations of culture management: Culture 1.0 is imposed from the top, Culture 2.0 encourages collaboration, and Culture 3.0 is created jointly by everyone in the organisation. Its principles are co-creation, autonomy and empowerment, transparency in decisions and communication, a shared purpose that goes beyond financial targets, and constant innovation in the culture itself.

## C8. Choosing a methodology

*In brief:* there is no universal answer. Choose, or combine, according to context, purpose and maturity, then test and adjust.

### Step 1: analyse the context

| Factor | Guidance |
|---|---|
| **Team size** | Small teams can work well with Scrum or XP. Large ones may need a scaling framework such as SAFe or LeSS. |
| **Nature of the product** | Products that are technically demanding, ship often and must be of high quality benefit from XP. Products subject to continual change and many inputs suit Kanban. |
| **Company culture** | Hierarchical organisations that resist change may need a gradual, educational start before complex frameworks. |
| **Level of uncertainty** | The more volatile the market or product, the stronger the case for a light, iterative approach. |
| **Team maturity** | Not every team is ready to self-organise or use advanced practices. Take earlier agile experience into account. |

### Step 2: define the main purpose

- Continuous delivery and avoiding blockages: Kanban.
- Building a close-knit team that delivers in iterations: start with Scrum.
- Better engineering practice and technical quality: XP.
- Other possible aims are a shorter time to market, closer business and technology collaboration, and a more adaptive, customer-centred culture.

### Step 3: choose one, or combine

| Combination | Suits |
|---|---|
| **Scrumban** | Scrum's structure with Kanban's flexibility, for teams that deliver often but need looser cycles. |
| **Scrum + XP** | Scrum's product and team management with XP's technical rigour. |
| **Kanban + Design Thinking** | Product discovery, where the emphasis is on fast exploration and validation. |
| **SAFe + Scrum** | Companies needing coordination among several Scrum teams within a strategic framework. |

Whatever is chosen should respect the Manifesto's values and make sense in context.

**Step 4: experiment, inspect and adapt**

Start with a pilot in one small team, review how well the method works at intervals, ask the team and stakeholders what they think, change practices, roles or even the framework when something is not working, and be willing to mix methods or move to another.

**The guide's four cases**

- A six-person start-up building a mobile app: Scrum, with XP practices added for code quality.
- A help desk: Kanban, because its work arrives continuously and unpredictably.
- A large multinational with many teams: SAFe, to scale Scrum and align on common goals.
- An innovation department: Design Thinking combined with agile, to discover and validate ideas before iterative development.

**Takeaway** Pick the method that fits the people, the goals and the environment, not the best-known name. Then keep adjusting it.

## C9. Structures that support agile at scale

*In brief:* supporting bodies and groupings that help many teams work in an agile way.

| Structure | Role |
|---|---|
| **Agile PMO** | A project management office that enables instead of controlling. It aligns teams' work with strategy, links teams so they do not work in silos, coaches, manages metrics as a basis for improvement and not for monitoring, and implements scaling frameworks in larger organisations. |
| **Agile Office** | The central point for supporting agile adoption: continuous coaching for teams and leaders, managing tools such as Jira, Trello and Monday.com, and promoting continuous improvement. |
| **Agile Transformation Team** | Agile experts, coaches and change leaders who lead the transformation: they set the strategy, facilitate cultural and structural change, and manage resistance. |
| **Agile Coaches** | Facilitators and mentors who guide and do not lead. They train teams and leaders, facilitate retrospectives and other key meetings, and help remove blockages. |
| **Chapters** | Groups of people with similar skills or roles, such as developers or designers, who share knowledge whichever team they work in. |
| **Guilds** | Informal networks across organisational boundaries for people interested in a particular topic. |
| **Communities of Practice** | Groups formed around a discipline or practice whose members meet regularly to share knowledge and build skill. |
| **Squads** | Small, autonomous, cross-functional teams, usually six to twelve people, each responsible for building, testing and launching one part of a product. |
| **Tribes** | Groups of squads working on related areas of a product, so they can coordinate and avoid duplication or integration problems. |

The guide attributes the tribes-and-squads structure to Spotify, which adopted it early.

### Business agility

Business agility is the capacity of the whole organisation, including marketing, finance, human resources and operations, to respond fast when markets shift, customers want something different or an opportunity appears, without interrupting the delivery of value. The guide names five components:

- **Agile culture** at every level, with readiness to change direction.
- **Distributed decision-making,** spread among teams instead of concentrated in senior management.
- **Customer focus** ahead of internal process and bureaucracy.
- **Constant innovation,** with new ideas tested and released regularly.
- **A flexible organisational structure:** fewer layers, cross-functional teams, agile processes.

## C10. Tools

*In brief:* software that helps teams visualise work, manage tasks, track progress and communicate.

| Tool | What it is | Features the guide highlights |
|---|---|---|
| **Jira** | Atlassian's tool for agile project management, especially in software, built to work with Scrum and Kanban. | Scrum and Kanban boards; backlog management with stories and subtasks; reports such as burndown, velocity and control charts; integrations with documentation, chat and CI/CD tools. |
| **Trello** | A simpler visual tool, also from Atlassian, based on boards, lists and cards. Suited to small teams and less complex projects. | Visual boards; labels and due dates; checklists and comments on cards; integrations and automation. |
| **Microsoft Teams** | A communication and collaboration platform. Not a project management tool in itself, but it integrates with them. | Channels and chat; integrations with Jira and Trello; video meetings for stand-ups, reviews and retrospectives; shared document editing; apps and bots. |
| **Monday.com** | A highly configurable project management tool adaptable to many kinds of project. | Customisable boards with Kanban, timeline and table views; custom automations; comments and file sharing; high-level progress tracking. |
| **Slack, Asana, ClickUp** | Mentioned briefly: a communication platform and two task and project management tools. | Slack integrates with tools such as Jira, Trello and GitHub. Asana and ClickUp offer boards, sprints, priorities and automation. |

### Reading notes (mine)

- **The text appears to be a translation,** with some odd wording. Besides “Read” for Lean and “People” for personas, one list of the values uses “about” where “over” is meant, and a “pilot” team appears once as a “driver”.
- **Its Scrum section is a general description, not the official rules.** For example it presents the three daily questions as standard, while the 2020 Scrum Guide leaves the format to the

Developers.

- **Its examples are illustrative.** The companies and figures in them are not sourced case studies.

# Part D: The three documents compared

This part is my own comparison. It matters for anyone answering questions from all three sources at once, because on some points they give different answers.

## D1. What each document covers

| Topic | Scrum Guide 2020 | Whitepaper | Methodologies Guide 2025 |
|---|---|---|---|
| **Agile Manifesto values** | Not covered | Listed | Listed and explained |
| **Waterfall and traditional project management** | Not covered | Covered | Mentioned briefly |
| **Scrum** | Full definition | Overview | Overview |
| **Kanban** | Not covered | Overview | Principles, metrics, roles |
| **Lean** | Named as a foundation | Seven software principles | Principles, seven wastes, tools |
| **Extreme Programming** | Not covered | Four practices in detail | Mentioned only |
| **Hybrid and Bimodal** | Not covered | Covered | Hybrid combinations only |
| **Design Thinking** | Not covered | Not covered | Covered |
| **Culture, transformation, scaling structures** | Not covered | Not covered | Covered |
| **Tools** | Not covered | Trello and CI tools named | Covered |

## D2. Where the accounts of Scrum differ

The Scrum Guide is the authority. The whitepaper follows the older 2016 edition, and the Methodologies Guide gives a general description.

| Point | Scrum Guide 2020 | Whitepaper | Methodologies Guide 2025 |
|---|---|---|---|
| **Sprint length** | One month or less | One to four weeks | Usually one to four weeks |
| **Team size** | Typically ten or fewer for the whole Scrum Team | Up to seven team members | Not stated |
| **Name for the builders** | Developers; no sub-teams | Scrum Team or Development Team | Development team |

| | | | |
|---|---|---|---|
| **How the team runs itself** | Self-managing | Self-organising | Self-managed |
| **Artifacts** | Product Backlog, Sprint Backlog, Increment, each with a commitment | Product Backlog, Sprint Backlog, Sprint Burndown Chart | Product Backlog and Sprint Backlog described; increment mentioned |
| **Burndown chart** | An optional forecasting practice | One of the three main artifacts | A transparency tool |
| **Sprint Planning** | Three topics: why, what, how | Two questions: what and how | Select items, define tasks, set the goal |
| **Daily Scrum format** | Any structure the Developers choose | Coordinate work and plan the next 24 hours | Three standard questions |
| **Product Owner** | A member of the Scrum Team; one person; accountable for value | Usually a customer or other stakeholder | Voice of the customer who manages the backlog |
| **Scrum Master** | Accountable for establishing Scrum and for team effectiveness; a leader who serves | Central figure who removes obstacles | Facilitator who protects the team |
| **Releasing** | Possible at any time in a Sprint; the review is not a gate | A potentially shippable increment at the end of each sprint | A potentially deliverable increment at the end of each sprint |
| **Product Goal and Definition of Done** | Both defined as commitments | Not mentioned | Not mentioned |

## D3. Two views of Lean

The two overview documents describe Lean from different directions, and both are valid.

- The **whitepaper** presents Lean software development: the Poppendiecks' seven principles, such as deciding late and delivering fast, and the minimum viable product.
- The **Methodologies Guide** presents Lean as it is used in manufacturing and operations: the seven wastes, kaizen, pull, and tools such as value stream mapping and 5S.
- They agree on the origin (Toyota), the central aim (remove waste, maximise customer value) and the importance of empowering the people who do the work.
- The Scrum Guide simply names lean thinking as one of Scrum's two foundations.

## D4. Which framework when

A combined view of the advice in Parts B and C.

| Framework | Good fit | Less suited to |
|---|---|---|
| **Scrum** | Complex products with changing requirements; a cohesive small team; regular stakeholder feedback. | Work that arrives unpredictably and cannot be planned a sprint at a time. |

| | | |
|---|---|---|
| **Kanban** | Continuous, unpredictable flows of small items such as support and maintenance; frequently changing priorities. | Teams that need the structure of fixed goals and roles. |
| **Lean** | Improving a whole process; small, short projects; minimum viable products. | Teams wanting a prescribed set of meetings and roles, since Lean provides none. |
| **XP** | Small teams where code quality and frequent delivery are critical and customers can work closely with developers. | Large teams, or teams without testing skills. |
| **Hybrid** | Products combining hardware and software; customers needing a fixed budget and date. | Settings where requirements are truly open-ended. |
| **Bimodal** | Organisations that must keep core systems stable while also innovating quickly. | Small organisations that cannot run two separate teams. |
| **Design Thinking** | Discovering what to build before building it. | Delivery itself; it is paired with another framework for that. |
| **Scaling frameworks (SAFe, LeSS)** | Many teams on one product in a large organisation. | A single small team. |

**Takeaway** For questions about Scrum's rules, trust the Scrum Guide. For breadth across frameworks, use the other two, remembering their age and wording.

# Quick reference

## Scrum at a glance (2020 Guide)

| Element | Contents |
|---|---|
| **Foundations** | Empiricism and lean thinking |
| **Three pillars** | Transparency, inspection, adaptation |
| **Five values** | Commitment, focus, openness, respect, courage |
| **Three accountabilities** | Developers, Product Owner, Scrum Master |
| **Five events** | The Sprint, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective |
| **Three artifacts** | Product Backlog, Sprint Backlog, Increment |
| **Three commitments** | Product Goal, Sprint Goal, Definition of Done |
| **Timeboxes for a one-month Sprint** | Planning 8 hours, Daily Scrum 15 minutes, Review 4 hours, Retrospective 3 hours |

## Glossary

| Term | Meaning |
|---|---|

| | |
|---|---|
| **Agile** | A set of values and principles for delivering value iteratively and adapting to change. An umbrella for many frameworks. |
| **Agile Manifesto** | The 2001 statement of four values and twelve principles written by seventeen software practitioners. |
| **Waterfall** | A linear model in which each project phase finishes before the next begins. |
| **Iteration / sprint** | A short, fixed-length cycle of work that ends with something usable. |
| **Increment** | A usable, verified addition to the product that meets the Definition of Done. |
| **Product Backlog** | The ordered list of everything needed to improve the product. |
| **Sprint Backlog** | The Sprint Goal, the items chosen for the Sprint, and the plan for delivering them. |
| **Definition of Done** | The agreed quality standard an item must meet to count as part of the Increment. |
| **Refinement** | Breaking down and clarifying backlog items so they are ready for a future Sprint. |
| **Empiricism** | Basing decisions on what has been observed. |
| **Kanban board** | A visual board showing work items moving through stages. |
| **WIP limit** | A cap on the number of items in progress at once. |
| **Cycle time / lead time** | Time from start to finish of a task, and time from request to delivery. |
| **Throughput** | Items completed in a given period. |
| **Scrumban** | A combination of Scrum's structure and Kanban's flow. |
| **Muda / kaizen** | Lean terms for waste and for continuous small improvement. |
| **MVP** | Minimum viable product: a release with just enough features to learn from early users. |
| **TDD** | Test-driven development: writing the test before the code. |
| **Refactoring** | Improving the structure of code without changing its behaviour. |
| **Continuous integration** | Merging and automatically testing code changes many times a day. |
| **Pair programming** | Two developers working together on the same code. |
| **Bimodal** | Running a predictable mode and an exploratory mode of IT work side by side. |
| **SAFe / LeSS** | Scaled Agile Framework and Large Scale Scrum: frameworks for coordinating many teams. |
| **Squad / tribe** | A small autonomous team, and a group of squads working on related parts of a product. |
| **Chapter / guild** | A group sharing a skill or role across teams, and an informal network around a topic of interest. |
| **Business agility** | The ability of a whole organisation to adapt quickly while continuing to deliver value. |

## Sources


- Schwaber, Ken, and Jeff Sutherland. *The Scrum Guide. The Definitive Guide to Scrum: The Rules of the Game*. November 2020. © 2020 Ken Schwaber and Jeff Sutherland. Licensed under Creative Commons Attribution Share-Alike 4.0 (creativecommons.org/licenses/by-sa/4.0).
- AltexSoft. *Agile Project Management: Best Practices and Methodologies*. Whitepaper, undated.
- EuropeanScrum.org. *Agile Methodologies Guide 2025,* version 1.0. Prepared by Yvonne Agnes, Silke Schöll and David Martí.


Page numbers given with quotations are the numbers printed on the pages.